

029 PyTorch迁移实验指南

用可复现检查区分正确迁移、梯度累加和意外断图

实验目标与准备

本实验沿用027的五条合成样本和26个参数，验证实际PyTorch CPU反向结果。产出不是“训练成功”的截图，而是一份逐坐标比较表、独立差分表及能重复触发的语义探针。先读正文第1至5节，准备能解释 $(5,2) @ (2,3) + (3,)$ 的广播行为。

在独立环境安装README中列出的Python、NumPy和CPU版PyTorch；不需要torchvision、torchaudio、CUDA或任何账户。environment.yml是学习者环境声明，不等于本课程测试过新的Anaconda安装。启动Jupyter后确认所选kernel属于这个环境；仅在另一个终端安装包不能改变当前Notebook内核。

在本讲目录运行：

```
python -c "import torch,numpy; print(torch.__version__, numpy.__version__)"
python experiment.py
python -m unittest -v test_experiment
```

脚本从自己的目录定位固定数据，不依赖当前工作目录。实验源码不会自动安装包或下载数据。缺少torch时先解决对应环境的依赖，不要删掉导入或改成一个假autograd类来获得“通过”。跨操作系统、GPU和浏览器Jupyter界面没有在本讲构建环境中实测。

任务一 建立数值与形状的检查顺序

打开data/batch.csv和data/config.json。写下五个标签、三层权重形状、每一层偏置形状和总参数数。程序固定报告入口会检查这两个文件的原始字节；不要直接覆盖它们来做新实验。你可以在新Python代码中构造数组并调用下列函数，输出保存到自己的新目录。

```
import experiment as e
import reference as r
X, y, layers, cfg = e.load_inputs()
expected = r.numpy_reference(X, y, layers)
actual = e.torch_result(X, y, layers)
rows = e.compare(expected, actual)
print(float(actual["loss"]), len(rows))
print(max(row["absolute_error"] for row in rows))
```

按comparisons.csv中的H0、Z1、H1、Z2、H2、Z3检查前向，再从dZ3向输入检查反向。dH0是输入梯度；index记录张量坐标，标量loss的index为空。

预期均值交叉熵约1.1489788656。比较表应全部通过混合容差。不要用这个约数和最终loss单独验收；任意一层形状错误、导数断路或倍率错误都应当被逐层检查发现。

任务二 独立手算并执行一个小层

使用正文的 X, W, b 和 $L = (1/2n) \sum Z_{ij}^2$ 。先手算 Z, L, dZ, dW, db, dX ，再运行：

```
import torch
x = torch.tensor([[1., 2.], [-1., 1.]],
                  dtype=torch.float64, requires_grad=True)
w = torch.eye(2, dtype=torch.float64, requires_grad=True)
b = torch.tensor([.5, -.5], dtype=torch.float64, requires_grad=True)
z = x @ w + b
z.retain_grad()
loss = .5 * z.square().sum() / len(x)
loss.backward()
print(loss.item(), z.grad, w.grad, b.grad, x.grad)
```

把`sum()/len(x)`改成`mean()`之前，先预测每一个输出。实验报告需解释多出来的输出维平均，不能只说“数值差一倍”。这个例子不使用交叉熵，目的是单独检验仿射层和广播反向。

任务三 比较两种独立数值证据

`reference.py`包含NumPy矩阵反向和另一个Python标量前向。后者不读取框架梯度，只用两个损失值做中心差分。`finite-difference.csv`检查26个参数加10个输入坐标，共36项。写出中心差分公式和步长，并说明为何不能只把步长缩得越小越好。

默认函数使用`tanh`。扩展实验可设置一个ReLU恰为0的点，比较框架局部规则与中心差分。应先承认它是不可微点，不能把差异简单归类为“PyTorch算错”。同时比较float32与float64迁移，保持数据、权重、损失和reduction不变；不要混合dtype后再把类型错误解释成数值误差。

任务四 重现三种断图表象

执行`e.semantic_probes(X,y,layers)`，查看`cuts`与`reconstructed_scalar_error`。所有预期异常都由探针捕获并记录，主程序应正常退出。不要要求每一个版本的英文报错字符串逐字相同，先核对异常类别和触发原因。

- 将标量结果重新构造成不需要梯度的tensor，再backward，立即报错。
- 在 H_1 后detach，下游参数仍然使loss需要梯度，反向能够完成，但第一层缺失。
- 在 H_1 后重建需要梯度的新tensor，只得到一个新的叶节点，旧第一层仍然缺失。

报告必须同时列出前向loss差、最后一层梯度差、输入及第一层梯度状态。修复后重新运行同一套检查。只验证loss相同或只验证`loss.requires_grad`均不够。

任务五 观察缓冲区与缓存的区别

读取accumulation，确认4、8、4的顺序。分别重写“每次重新前向但忘记清梯度”和“同一个平方loss重复backward”的程序，预测前者累加、后者因缓存释放报错。再看microbatch，解释加权误差接近舍入量而等权误差约0.0330774。

把参数更新放在所有微批量梯度计算之后。使用with torch.no_grad()执行普通更新，然后重新前向；不要保留旧loss继续反向。记录更新前后损失，并说明它没有证明什么。

任务六 原地操作与模式开关

在新局部张量上分别重现叶节点add_、tanh保存结果add_、detach别名add_、no_grad内先改参数再对旧loss反向。每次使用新变量，不要让一个失败图污染下一次诊断。再观察独立NumPy别名探针中前向值4与梯度6的矛盾；修复版复制输入后梯度应回到4。

检查eval和no_grad输出。Dropout的 $p = 1$ 仅用于稳定教学对照。填写四种组合：训练/评估模式乘以启用/禁用梯度记录。补充“工厂函数显式requires_grad”的例外，并解释输入参数原标志为何仍然保留。

提交物与重跑要求

提交手算过程、全部逐层比较结论、36坐标差分摘要、三个断图诊断、四类原地错误和eval/no_grad对照。运行Notebook前清空输出并重启内核，从上到下执行。Notebook最后会实际执行测试，不能靠保存的旧输出证明成功。

默认脚本写入五个文件：summary.json、comparisons.csv、finite-difference.csv、semantic-probes.json、tensors.json。Notebook写到单独的notebook_outputs，应与同环境脚本结果一致。版本变更后先核查语义和容差，不要求跨库版本逐字节一致。保存目录的替换是单文件级，遇到操作系统写入故障不承诺五个文件整体事务原子性。